

Antworten zu Blatt 11

Nr. 1:

WEAK SCALING:

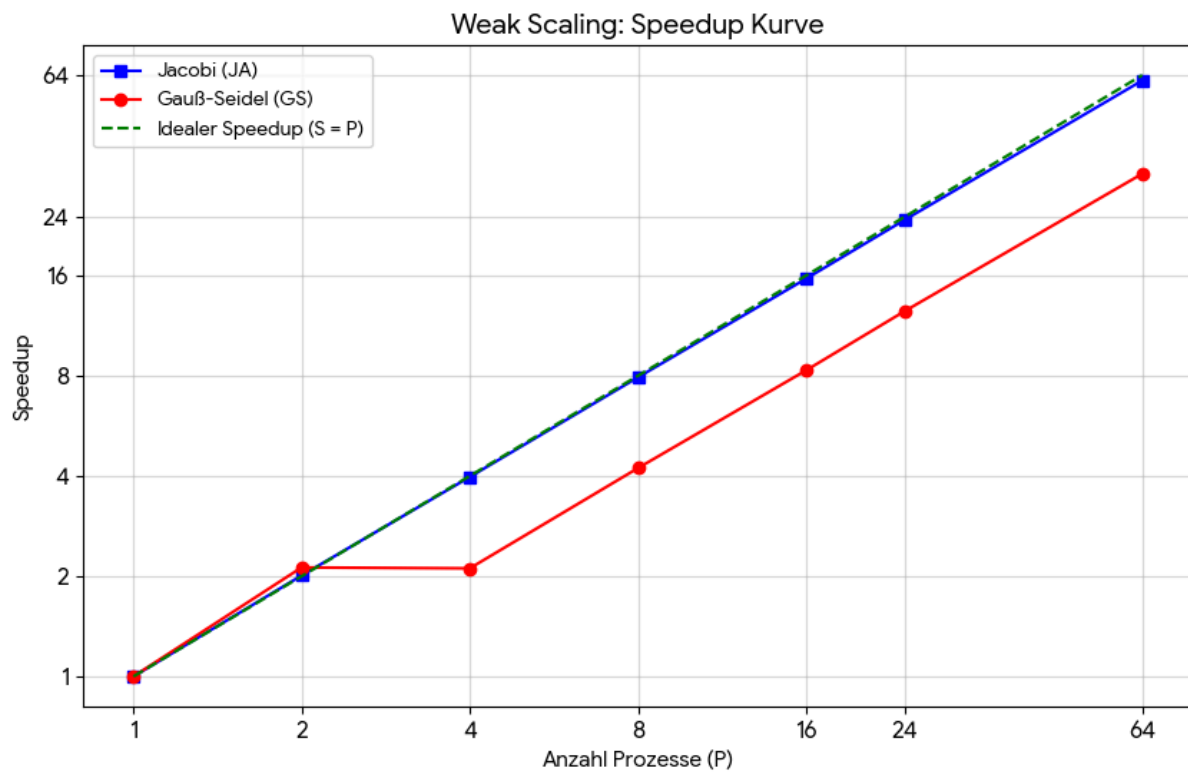
NPROCS	NNODES	ILINES	TIME_JA	TIME_GS
1	1	400	251.51	270.58
2	1	564	250.25	254.62
4	2	800	253.23	512.49
8	4	1128	254.29	512.46
16	4	1600	257.02	521.20
24	4	1960	256.96	519.63
64	8	3200	260.92	535.02



Wie erklären Sie sich die Wahl der Interlines in Bezug auf die Prozesszahl?

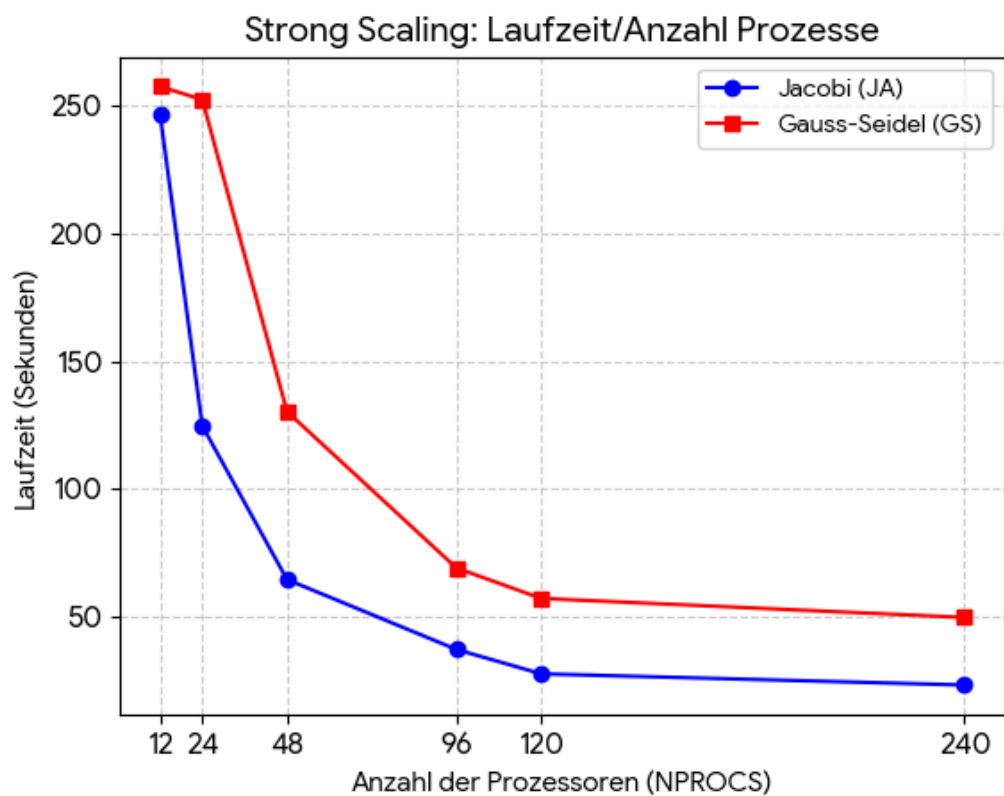
Die Interlines wird bei Weak Scaling so gewählt, dass bei Skalierung die Problemgröße pro Prozess gleich bleibt. Da die Interlines ein Faktor für beide Dimensionen der Matrix ist, steigt die Prozessanzahl um vierfache bei einer Verdoppelung der Problemgröße (bestimmt durch Interlines).

Wie sähe die Speedupkurve bei idealem Weak Scaling aus?

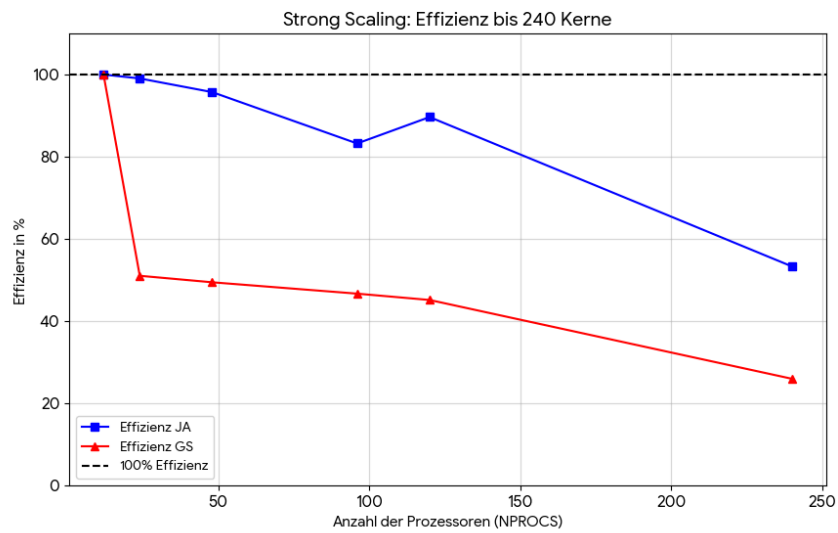
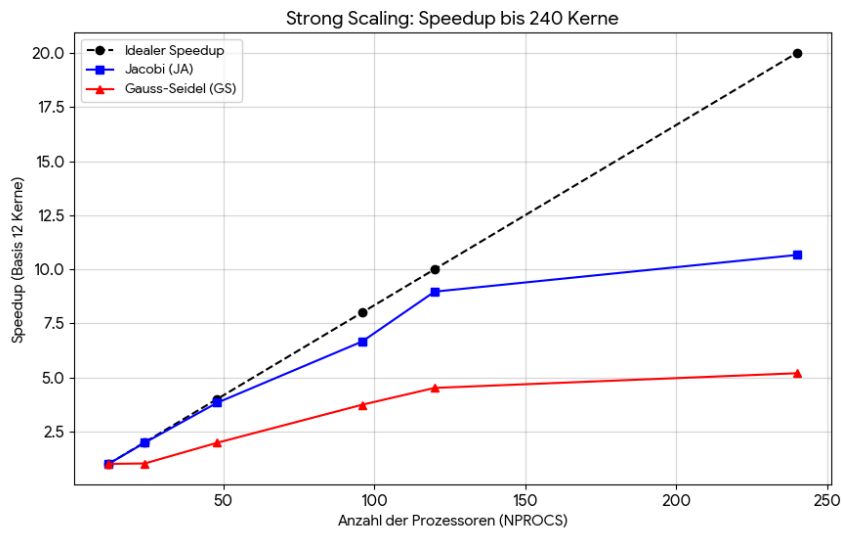


STRONG SCALING:

NPROCS	NNODES	ILINES	TIME_JA	TIME_GS
12	1	1920	246.80	257.53
24	2	1920	124.59	252.38
48	4	1920	64.46	130.23
96	8	1920	37.07	68.97
120	10	1920	27.53	57.06
240	10	1920	23.15	49.60

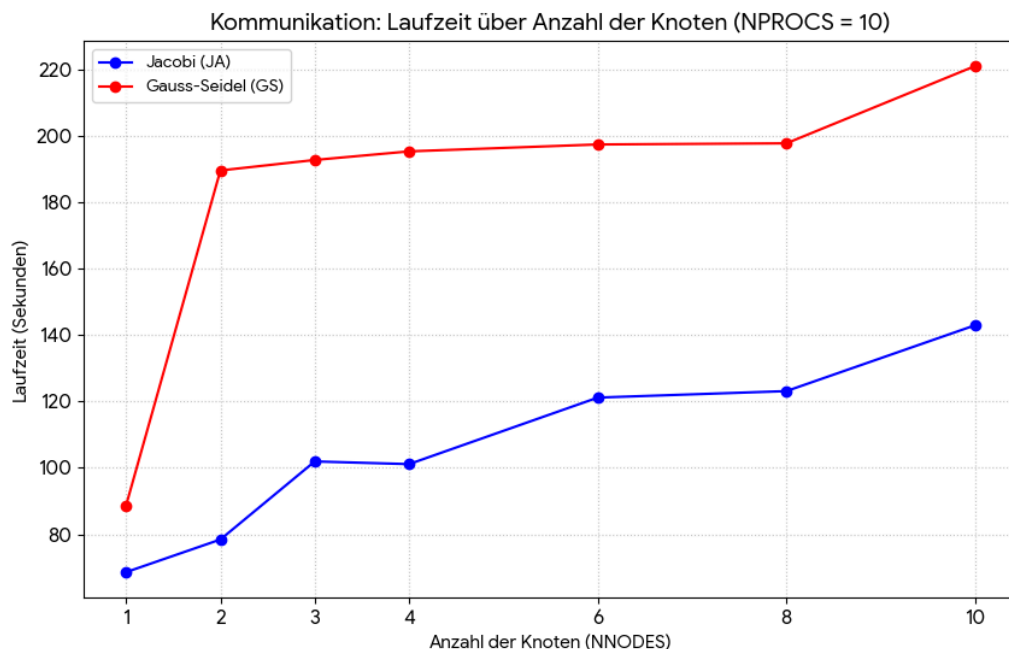


Wie sähe die Speedupkurve bzw. Effizienzkurve bei idealem Strong Scaling aus?



COMMUNICATION:

NPROCS	NNODES	ILINES	TIME_JA	TIME_GS
10	1	200	68.62	88.68
10	2	200	78.45	189.50
10	3	200	101.94	192.65
10	4	200	101.13	195.24
10	6	200	121.15	197.32
10	8	200	123.10	197.66
10	10	200	142.91	220.91

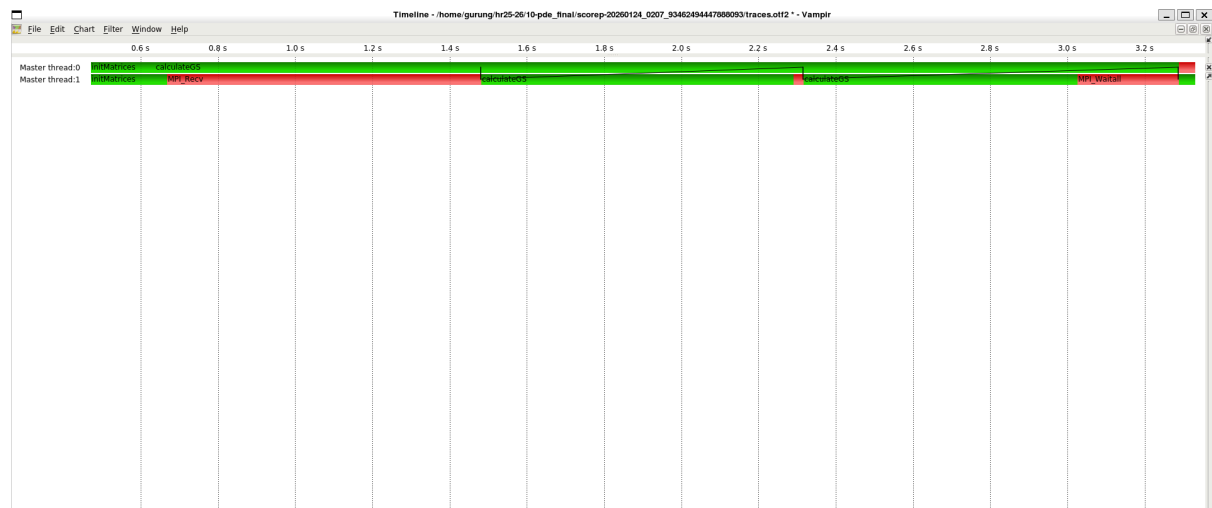


Gibt es Auffälligkeiten? Wie erklären Sie sich diese?

Im Gegensatz zum Strong bzw. Weak Scaling wurde die Problemgröße bzw. die Anzahl der Prozesse zum parallelen Rechnen nicht erhöht. Dennoch ist die Laufzeit gestiegen, was anhand der Auswertung auf die Anzahl der Rechnerknoten zurückzuführen ist. Erklären lässt sich dies über den erhöhten Kommunikationsaufwand zwischen Knoten (Latenz, Bandbreite, Overhead)

Nr. 2

Zeit	Rang 0	Rang 1
~ 0.0000s	MPI_Bcast	MPI_Bcast
0.5355s		MPI_Recv (A)
1.3478s	MPI_Issend	
1.3481s	MPI_Issend, MPI_Waitall (B)	MPI_Recv (A), MPI_Recv (in/out)
1.3482s	MPI_Waitall (B)	
1.3485s		MPI_Issend
2.1389s	MPI_Recv (in/out)	
2.1392s	MPI_Issend	
2.1394s	MPI_Issend, MPI_Waitall (C)	
2.1452s		MPI_Waitall (in/out)
2.1453s	MPI_Waitall (C)	MPI_Recv (in/out), MPI_Recv (in/out)
2.1456s		MPI_Issend
2.8485s		MPI_Waitall (D)
2.9679s	MPI_Recv (in/out)	MPI_Waitall (D)
2.9680s		MPI_Recv (E)
2.9682s	MPI_Issend	
2.9684s	MPI_Issend, MPI_Waitall (in/out)	MPI_Recv (E), MPI_Recv (in/out)
3.7783s		MPI_Waitall (in/out)



(ursprüngliche programm)

Wenn wir die Vampir Ausgabe und das manuelle Profiling gegenüberstellen, erkenne wir direkt Gemeinsamkeiten in der Kommunikationsdauer. Zuerst ist ein auffällig langer MPI_Recv von Prozess 1 zu sehen, da dieser warten muss, bis Prozess 0 eine ganze Iteration auf seiner Matrix Allokierung durchgeführt hat. Dies dauerte bei beiden Läufen ca. 0.8 Sekunden. Die MPI_Waitall von Prozess 0 sind in Vampir kaum erkennbar: sie liefen extrem kurz. Das liegt daran, dass Prozess 1 schon in dem MPI_Recv Block ist. Prozess 1 schickt nach Berechnung der ersten Zeile diese hoch zu Prozess 0. Dies läuft wieder extrem kurz ($6.4 \cdot 10^{-5}$ Sekunden). Die Kommunikation läuft hier wie im Schema zu Blatt 9 weiter.

Im Allgemeinen ist der Programmablauf bei beidem gleich.

Unterschiede sind lediglich in der Abstraktion und der Erschließbarkeit der Daten zu finden.

Bei der Vampir Ausgabe erkennt man eindeutig die Kommunikationsrichtung, aber bei den Prints wird das nicht sofort eindeutig und man muss gewandter in der Systemstruktur des vorliegenden Programms sein.